# CASE TOOLS
# &
# DATA FLOW DIAGRAM (DFD)

# Introduction

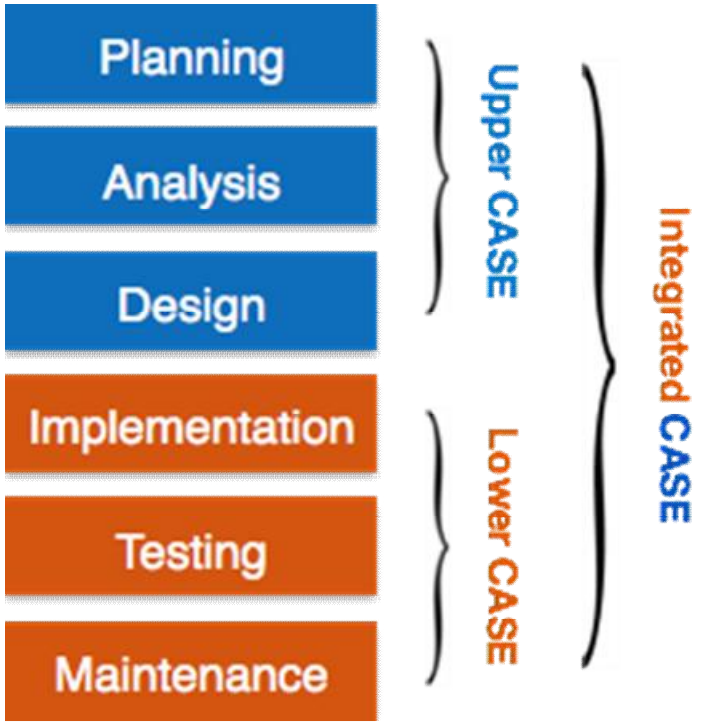
- CASE stands for **C**omputer **A**ided **S**oftware **E**ngineering. It means, development and maintenance of software projects with help of various automated software tools.
- CASE Tools
- CASE tools are set of software application programs, which are used to automate SDLC activities. CASE tools are used by software project managers, analysts and engineers to develop software system.
- There are number of CASE tools available to simplify various stages of Software Development Life Cycle such as Analysis tools, Design tools, Project management tools, Database Management tools, Documentation tools are to name a few.
- Use of CASE tools accelerates the development of project to produce desired result and helps to uncover flaws before moving ahead with next stage in software development.

# Component of CASE Tools

CASE tool is composed of:

- **Central Repository** - CASE tools require a central repository, which can serve as a source of common, integrated and consistent information. Central repository is a central place of storage where **product specifications, requirement documents, related reports and diagrams, other useful information regarding management is stored.** Central repository also serves as **data dictionary.**

# Categories of CASE Tools




- **Upper Case Tools** - Upper CASE tools are used in planning, analysis and design stages of SDLC.
- **Lower Case Tools** - Lower CASE tools are used in implementation, testing and maintenance.
- **Integrated Case Tools** - Integrated CASE tools are helpful in all the stages of SDLC, from Requirement gathering to Testing and documentation.

# CASE Tools

- CASE tools can be grouped together if they have similar functionality, process activities and capability of getting integrated with other tools.
- **Scope of Case Tools**
- The scope of CASE tools goes throughout the SDLC.

# Case Tools Types

**Diagram tools**

- These tools are used to represent system components, data and control flow among various software components and system structure in a graphical form. *For example, Flow Chart Maker tool for creating state-of-the-art flowcharts.*

**Process Modeling Tools**

- Process modeling is method to create software process model, which is used to develop the software. Process modeling tools help the managers to choose a process model or modify it as per the requirement of software product. *For example, EPF Composer*

# Case Tools Types

## Project Management Tools

- These tools are used for project planning, cost and effort estimation, project scheduling and resource planning. Managers have to strictly comply project execution with every mentioned step in software project management. Project management tools help in storing and sharing project information in real-time throughout the organization. *For example, Creative Pro Office, Trac Project, Basecamp.*

# Case Tools Types

## Documentation Tools

- Documentation in a software project starts prior to the software process, goes throughout all phases of SDLC and after the completion of the project.
- Documentation tools generate documents for technical users and end users. Technical users are mostly in-house professionals of the development team who refer to system manual, reference manual, training manual, installation manuals etc. The end user documents describe the functioning and how-to of the system such as user manual. *For example, Doxygen, DrExplain, Adobe RoboHelp for documentation.*

# Case Tools Types

**Analysis Tools**

- These tools help to gather requirements, automatically check for any inconsistency, inaccuracy in the diagrams, data redundancies or erroneous omissions. *For example, Accept 360, Accompa, CaseComplete for requirement analysis, Visible Analyst for total analysis.*

**Design Tools**

- These tools help software designers to design the block structure of the software, which may further be broken down in smaller modules using refinement techniques. These tools provides detailing of each module and interconnections among modules. *For example, Animated Software Design*

# Case Tools Types

**Configuration Management Tools**

- An instance of software is released under one version. Configuration Management tools deal with:
- Version and revision management
- Baseline configuration management
- Change control management
- CASE tools help in this by automatic tracking, version management and release management. **For example, Fossil, Git, Accu REV.**

**Change Control Tools**

- These tools are considered as a part of configuration management tools. They deal with changes made to the software after its baseline(Reference point) is fixed or when the software is first released. CASE tools automate change tracking, file management, code management and more. It also helps in enforcing change policy of the organization.

# Case Tools Types

**Programming Tools**

- These tools consist of programming environments like IDE (Integrated Development Environment), in-built modules library and simulation tools. These tools provide comprehensive aid in building software product and include features for simulation and testing. *For example, Cscope to search code in C, Eclipse.*

**Prototyping Tools**

- Software prototype is simulated version of the intended software product. Prototype provides initial look and feel of the product and simulates few aspect of actual product.
- Prototyping CASE tools essentially come with graphical libraries. They can create hardware independent user interfaces and design. These tools help us to build rapid prototypes based on existing information. In addition, they provide simulation of software prototype. *For example, Serena prototype composer, Mockup Builder.*

# Case Tools Types

## Web Development Tools

- These tools assist in designing web pages with all allied elements like forms, text, script, graphic and so on. Web tools also provide live preview of what is being developed and how will it look after completion. *For example, Fontello, Adobe Edge Inspect, Foundation 3, Brackets.*

## Quality Assurance Tools

- Quality assurance in a software organization is monitoring the engineering process and methods adopted to develop the software product in order to ensure conformance of quality as per organization standards. QA tools consist of configuration and change control tools and software testing tools. *For example, SoapTest, AppsWatch, JMeter.*

# Case Tools Types

## Maintenance Tools

- Software maintenance includes modifications in the software product after it is delivered. Automatic logging and error reporting techniques, automatic error ticket generation and root cause Analysis are few CASE tools, which help software organization in maintenance phase of SDLC. *For example, Bugzilla for defect tracking, HP Quality Center.*

# DATA FLOW DIAGRAM (DFD)

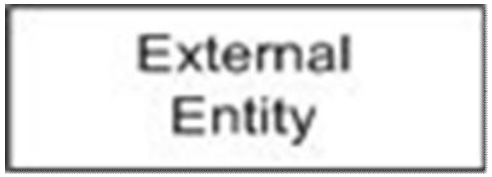
# Introduction

- **Data flow diagram** is one of the tools used in the analysis phase.
- **A Data Flow Diagram (DFD) is used** to describe the logical operation of a system, i.e. what a system does. It depicts the flow of data through the system and the work or processing performed by that system.
- Data flow diagram is a graphical tool used to analyze the movement of data through a system-manual or automated-including the processes, stores of data, and delay in the system.
- The transformation of data from input to output, through processes, may be described logically and independently of the physical components (for example, computers, file cabinets, disk units, and word processors).
- ***Structure chart*** is another tool used in designing phase of the life cycle.

# Elements of Data Flow Diagrams

- Data Flow Diagrams are composed of the four basic symbols shown below.

i. The ***External Entity symbol*** represents sources of data to the system or destinations of data from the system. External entities determine the system boundary. They are external to the system being studied. They are often beyond the area of influence of the developer.



# Elements of Data Flow Diagrams

ii. The ***Data Flow symbol*** represents movement of data. That is represents the input (or output) of data to (or from) a process ("data in motion").



# Elements of Data Flow Diagrams

iii. The ***Data Store symbol*** represents data that is not moving (delayed data at rest). Data Stores are repository for data that are temporarily or permanently recorded within the system. It is an "inventory" of data. These are common link between data and process models. Only processes may connect with data stores.

or

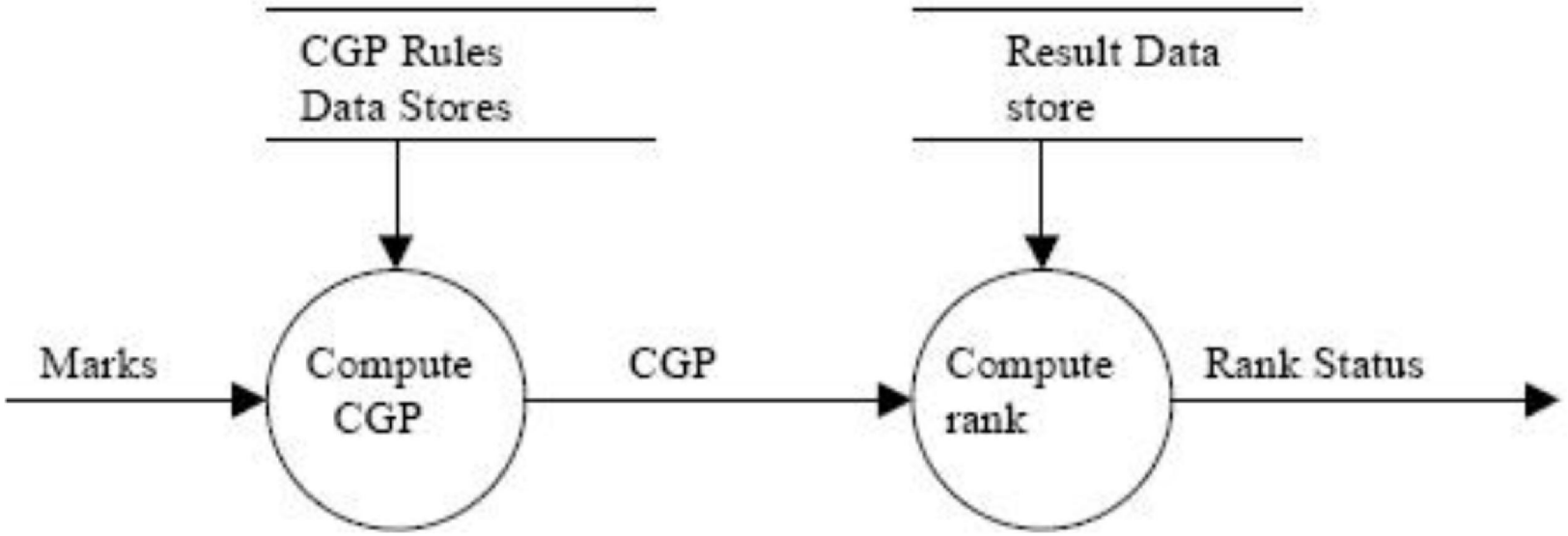


# Elements of Data Flow Diagrams

iv. The ***Process symbol*** represents an activity that transforms or manipulates the data (combines, reorders, converts, etc.). Processes are work or actions performed on incoming data flows to produce outgoing data flows. These show data transformation or change.



# An Example of a DFD showing rank calculation process of a university



# Different Levels of DFDs

- The DFD may be used to represent a system or software at any level of abstraction.
- In fact, DFDs may be partitioned into levels that represent increasing information flow and functional details.
- A ***0 level DFD***, also called a **context model**, represents the entire software elements as a single bubble with input and output data indicated by incoming and outgoing arrows, respectively.
- Additional processes and information flow paths are represented as the 0 level DFD is partitioned to reveal more information.
- For example, a ***1 level DFD*** might contain five or six bubbles with interconnecting arrows.

# Making DFDs (data Flow Diagrams)

- Data Flow Diagramming is a means of representing a system at any level of detail with a graphic network of symbols showing data flows, data stores, data processes, and data sources/destinations.
- The ***goal*** of data flow diagramming is to have a commonly understood model of a system.
- The diagrams are the basis of structured systems analysis. Data flow diagrams are supported by other techniques of structured systems analysis such as data structure diagrams, data dictionaries, and procedure-representing techniques such as decision tables, decision trees, and structured English.
- The purpose of data flow diagrams is to provide a semantic bridge between users and systems developers. The diagrams are graphical, *eliminating thousands of words.*
- These are logical representations, modeling what a system does, rather than physical models showing how it does it.
- DFDs are hierarchical, showing systems at any level of detail. Finally, it should be jargonless, allowing user understanding and reviewing.

- Also, data flow diagrams have the objective of avoiding the cost of:
  - user/developer misunderstanding of a system, resulting in a need to redo systems or in not using the system.
  - having to start documentation from scratch when the physical system changes since the logical system, WHAT gets done, often remains the same when technology changes.
  - systems inefficiencies because a system gets "computerized" before it gets "systematized".
  - being unable to evaluate system project boundaries or degree of automation, resulting in a project of inappropriate scope.